Signal System Properties

Linearity • Causality • Time-Invariance

Concepts, Code & Syntax with NumPy

This reference covers the three core system properties for a signals coding exam. Each has its definition, the mathematical test, a reusable NumPy checker, and usage. All checkers take a `system` function and return `True` / `False`.

1. Common Setup & NumPy Syntax

Every checker relies on the same handful of NumPy tools. Learn these first.

```
import numpy as np

# --- Generating test signals ---
n  = np.arange(0, 64)              # index vector 0..63
x  = np.random.default_rng(0).standard_normal(64)  # random signal (seeded)
x1 = np.sin(0.2 * n)               # a sinusoid
x2 = np.cos(0.1 * n)               # another sinusoid

# --- Comparing floats (NEVER use ==) ---
np.allclose(a, b, atol=1e-9)     # True if arrays match within tolerance

# --- Copying (avoid shared memory) ---
b = a.copy()                       # independent copy

# --- Shifting / delaying a signal ---
np.roll(x, 1)      # shift RIGHT -> x[n-1]  (delay / past)
np.roll(x, -1)     # shift LEFT  -> x[n+1]  (advance / future)

# --- Reversing (time reversal x[-n]) ---
x[::-1]            # reversed view (add .copy() for independence)
```

Defining a system

A "system" is just a function mapping input array to output array. Two ways:

```
def amp(x):           # explicit form
    return 2 * x

amp = lambda x: 2*x  # one-line form (used in examples below)
```

2. Linearity

Definition. A system is **linear** if it obeys *superposition* — additivity plus scaling (homogeneity):

$$a_1 x_1[n] + a_2 x_2[n] \longrightarrow a_1\, y_1[n] + a_2\, y_2[n]$$

Test. Combine two inputs with random scalars, then check both orders match:

$$\underbrace{\text{system}(a x_1 + b x_2)}_{\text{combine, then process}} = \underbrace{a\,\text{system}(x_1) + b\,\text{system}(x_2)}_{\text{process, then combine}}$$

A necessary sign: **zero input must give zero output** (so $y = 2x + 3$ is *not* linear).

```
def is_linear(system, N=64, trials=5, tol=1e-9):
    rng = np.random.default_rng(0)
    for _ in range(trials):
        x1, x2 = rng.standard_normal(N), rng.standard_normal(N)
        a, b   = rng.standard_normal(), rng.standard_normal()
        lhs = system(a*x1 + b*x2)                # combine then process
        rhs = a*system(x1) + b*system(x2)        # process then combine
        if not np.allclose(lhs, rhs, atol=tol):
            return False
    return True
```

Usage:

```
is_linear(lambda x: 2*x)          # True  - amplifier
is_linear(lambda x: np.cumsum(x)) # True  - integrator
is_linear(lambda x: x**2)         # False - squarer
is_linear(lambda x: np.abs(x))    # False - absolute value
is_linear(lambda x: 2*x + 3)      # False - affine (the trap)
```

3. Causality

Definition. A system is **causal** if the output depends only on **present and past** inputs, never the future:

$$y[n_0] \text{ depends only on } x[n] \text{ for } n \leq n_0.$$

For an LTI system this means the impulse response satisfies $h[n] = 0$ for $n < 0$.

Test. Make two inputs identical in the past but **different in the future**. A causal system's past/present output must not change.

```
def is_causal(system, N=64, trials=5, tol=1e-9):
    rng = np.random.default_rng(0)
    for _ in range(trials):
        x1 = rng.standard_normal(N)
        x2 = x1.copy()
        split = N // 2
        x2[split:] = rng.standard_normal(N - split)  # change future only
        y1, y2 = system(x1), system(x2)
        # past/present outputs must stay identical
        if not np.allclose(y1[:split], y2[:split], atol=tol):
            return False
    return True
```

Usage:

```
is_causal(lambda x: np.roll(x, 1))   # True  - delay x[n-1] (past)
is_causal(lambda x: np.cumsum(x))    # True  - integrator
is_causal(lambda x: np.roll(x, -1))  # False - advance x[n+1] (future)
is_causal(lambda x: x[::-1])         # False - time reversal
```

Remember the roll direction

`np.roll(x, +1)` $\Rightarrow x[n-1]$ (past, causal). `np.roll(x, -1)` $\Rightarrow x[n+1]$ (future, non-causal).

4. Time-Invariance

Definition. A system is **time-invariant** if a shift in the input causes the *same* shift in the output — its behavior doesn't change over time:

$$x[n] \to y[n] \quad \Longrightarrow \quad x[n-k] \to y[n-k].$$

Visual clue: if the equation contains an explicit n or t outside x (e.g. $y = n\,x[n]$), it is time-*varying*.

Test. Compare **shift-then-process** against **process-then-shift**. Ignore the first k samples (edge wrap from `np.roll`).

```
def is_time_invariant(system, N=64, trials=5, k=5, tol=1e-9):
    rng = np.random.default_rng(0)
    for _ in range(trials):
        x = rng.standard_normal(N)
        pathA = system(np.roll(x, k))     # shift input, then process
        pathB = np.roll(system(x), k)     # process, then shift output
        if not np.allclose(pathA[k:], pathB[k:], atol=tol):  # skip wrap edge
            return False
    return True
```

Usage:

```
is_time_invariant(lambda x: 2*x)                    # True  - fixed gain
is_time_invariant(lambda x: x**2)                   # True  - nonlinear but TI
is_time_invariant(lambda x: np.arange(len(x)) * x)  # False - n*x[n]
is_time_invariant(lambda x: x[::-1])                # False - time reversal
```

5. Putting It Together: Classify a System

Linearity, causality, and time-invariance are **independent** properties. A system can have any combination. Linear + Time-Invariant = **LTI**, the class where convolution and transforms apply.

```
def classify(system, name=""):
    L  = is_linear(system)
    TI = is_time_invariant(system)
    C  = is_causal(system)
    tag = " -> LTI" if (L and TI) else ""
    print(f"{name:12s} Linear={L}  TimeInv={TI}  Causal={C}{tag}")

classify(lambda x: np.cumsum(x),            "cumsum")   # L, TI, C -> LTI
classify(lambda x: x**2,                    "square")   # nonlinear, TI, C
classify(lambda x: np.arange(len(x)) * x,   "n*x[n]")   # linear, time-varying
classify(lambda x: np.roll(x, -1),          "advance")  # linear, TI, non-causal
```

System	Linear	Time-Inv.	Causal
$y = 2x[n]$	Yes	Yes	Yes
$y = x^2[n]$	No	Yes	Yes
$y = n\,x[n]$	Yes	No	Yes
$y = x[n+1]$	Yes	Yes	No
$y = 2x[n] + 3$	No	Yes	Yes
$y = x[-n]$	Yes	No	No

Exam survival tips

- Always use `np.allclose`, never `==`, to compare float arrays.
- Keep the system as a **function** so you can swap it into any checker.
- Watch the traps: `2*x+3` (not linear), `np.abs` (not linear), time reversal (not causal, not TI), explicit `n` (not TI).
- Seed the RNG (`default_rng(0)`) for reproducible results.